

Exercise 4

Writing a Simple Linux Character Device Driver

Kernel module basics: registration, file operations, and the user ↔ kernel boundary

Host OS: Ubuntu Linux | Kernel: 6.19.14

Contents

1. Environment
 2. Create the Project Directory
 3. Write the Driver Source (mychardev.c)
 4. Write the Makefile
 5. Build the Driver
 6. Load the Driver
 7. Check the Kernel Messages
 8. Check /dev
 9. First Test — Write Data
 10. Read the Data Back
 11. What Actually Happened?
 12. The Four Important Functions
 13. Simple Diagram for the Class
 14. What Students Learn From This Experiment
 15. Student Exercises
- Conclusion

1. Environment

Kernel: 6.19.14

Install the build tools needed to compile a kernel module against the running kernel's headers:

```
sudo apt update
sudo apt install build-essential linux-headers-$(uname -r)
```

2. Create the Project Directory

```
mkdir simple_char_driver
cd simple_char_driver
```

Two files are used in this exercise:

```
simple_char_driver/
├── mychardev.c
└── Makefile
```

3. Write the Driver Source (mychardev.c)

Create the source file:

```
nano mychardev.c
```

The driver registers a character device and implements four callbacks — open, read, write, and release — backed by a small fixed-size in-kernel buffer:

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/cdev.h>
#include <linux/device.h>

#define DEVICE_NAME "mychardev"
#define CLASS_NAME "mycharclass"
#define BUFFER_SIZE 1024

static dev_t dev_num;
static struct cdev my_cdev;
static struct class *my_class;
static struct device *my_device;
static char kernel_buffer[BUFFER_SIZE];
static size_t data_size;

/* open() */
static int my_open(struct inode *inode, struct file *file)
{
    pr_info("mychardev: device opened\n");
    return 0;
}

/* read() */
static ssize_t my_read(struct file *file, char __user *user_buffer,
                      size_t count, loff_t *offset)
{
    size_t bytes_to_copy;

    if (*offset >= data_size)
        return 0;

    bytes_to_copy = min(count, data_size - (size_t)*offset);
```

```

    if (copy_to_user(user_buffer, kernel_buffer + *offset, bytes_to_copy))
        return -EFAULT;

    *offset += bytes_to_copy;
    pr_info("mychardev: read %zu bytes\n", bytes_to_copy);
    return bytes_to_copy;
}

/* write() */
static ssize_t my_write(struct file *file, const char __user *user_buffer,
                        size_t count, loff_t *offset)
{
    size_t bytes_to_copy;

    bytes_to_copy = min(count, (size_t)(BUFFER_SIZE - 1));

    if (copy_from_user(kernel_buffer, user_buffer, bytes_to_copy))
        return -EFAULT;

    kernel_buffer[bytes_to_copy] = '\0';
    data_size = bytes_to_copy;
    pr_info("mychardev: received %zu bytes: %s\n",
            bytes_to_copy, kernel_buffer);
    return bytes_to_copy;
}

/* release() */
static int my_release(struct inode *inode, struct file *file)
{
    pr_info("mychardev: device closed\n");
    return 0;
}

/* File operations */
static const struct file_operations my_fops = {
    .owner    = THIS_MODULE,
    .open     = my_open,
    .read     = my_read,
    .write    = my_write,
    .release  = my_release,
};

/* Module initialization */
static int __init my_driver_init(void)
{
    int ret;

    pr_info("mychardev: initializing driver\n");

    /* Allocate major/minor number */
    ret = alloc_chrdev_region(&dev_num, 0, 1, DEVICE_NAME);
    if (ret < 0) {
        pr_err("mychardev: failed to allocate device number\n");
        return ret;
    }
    pr_info("mychardev: major = %d, minor = %d\n",
            MAJOR(dev_num), MINOR(dev_num));

    /* Initialize and add cdev */
    cdev_init(&my_cdev, &my_fops);
    my_cdev.owner = THIS_MODULE;

```

```

ret = cdev_add(&my_cdev, dev_num, 1);
if (ret < 0) {
    pr_err("mychardev: failed to add cdev\n");
    unregister_chrdev_region(dev_num, 1);
    return ret;
}

/* Create device class and device node */
my_class = class_create(CLASS_NAME);
if (IS_ERR(my_class)) {
    cdev_del(&my_cdev);
    unregister_chrdev_region(dev_num, 1);
    return PTR_ERR(my_class);
}

my_device = device_create(my_class, NULL, dev_num, NULL, DEVICE_NAME);
if (IS_ERR(my_device)) {
    class_destroy(my_class);
    cdev_del(&my_cdev);
    unregister_chrdev_region(dev_num, 1);
    return PTR_ERR(my_device);
}

pr_info("mychardev: driver loaded successfully\n");
pr_info("mychardev: device created at /dev/%s\n", DEVICE_NAME);
return 0;
}

/* Module cleanup */
static void __exit my_driver_exit(void)
{
    device_destroy(my_class, dev_num);
    class_destroy(my_class);
    cdev_del(&my_cdev);
    unregister_chrdev_region(dev_num, 1);
    pr_info("mychardev: driver unloaded\n");
}

module_init(my_driver_init);
module_exit(my_driver_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Linux Kernel Lab");
MODULE_DESCRIPTION("Simple Character Device Driver");
MODULE_VERSION("1.0");

```

4. Write the Makefile

```

nano Makefile
obj-m += mychardev.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean

```

5. Build the Driver

```
make
```

This produces, alongside the existing source files:

- mychardev.o — compiled object file
- mychardev.ko — the loadable kernel module (the important file)

6. Load the Driver

Load the module:

```
sudo insmod mychardev.ko
```

Confirm it's loaded:

```
lsmod | grep mychardev
```

Expected output:

```
mychardev                16384  0
```

7. Check the Kernel Messages

```
dmesg | tail
```

Expected output:

```
mychardev: initializing driver
mychardev: major = 240, minor = 0
mychardev: driver loaded successfully
mychardev: device created at /dev/mychardev
```

8. Check /dev

```
ls -l /dev/mychardev
```

Expected output:

```
crw----- 1 root root 240, 0 ... /dev/mychardev
```

The leading c is the important detail — it marks /dev/mychardev as a character device, as opposed to a block device (b) or a regular file (-).

9. First Test — Write Data

The driver can be tested directly from the shell, without a separate test application:

```
echo "Hello Kernel" | sudo tee /dev/mychardev
```

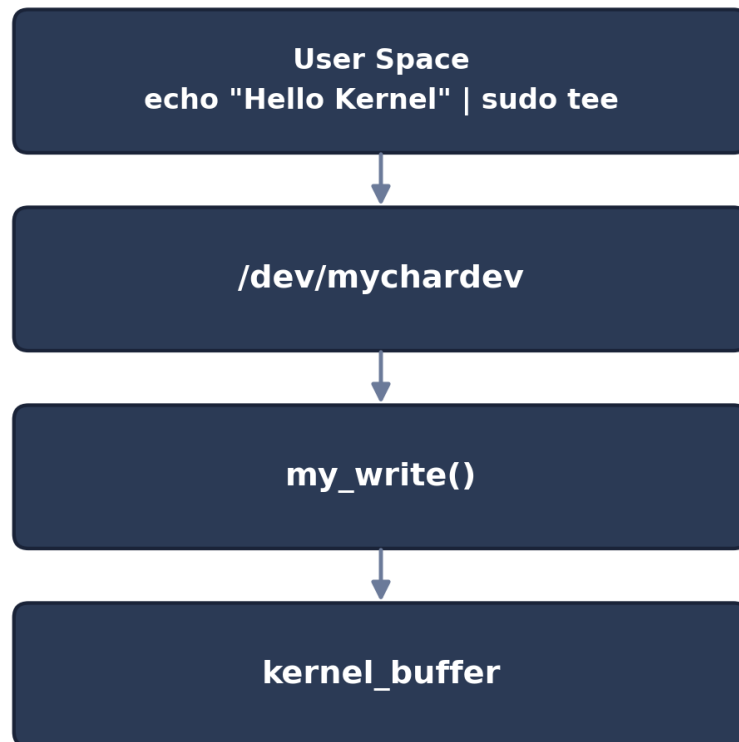


Figure 1 — Path of the data from the shell into the driver's write handler.

Check the kernel log:

```
dmesg | tail
```

Expected output:

```
mychardev: device opened
mychardev: received 13 bytes: Hello Kernel
```

10. Read the Data Back

```
sudo cat /dev/mychardev
```

Expected output:

```
Hello Kernel
```

11. What Actually Happened?

This is the core of the exercise: understanding how data crosses the user/kernel boundary.

Writing: user space → kernel space

When `echo "Hello Kernel" | sudo tee /dev/mychardev` ran, the driver's `my_write()` was called. Inside it, `copy_from_user()` moved the string from the user-space buffer into the kernel-space buffer:

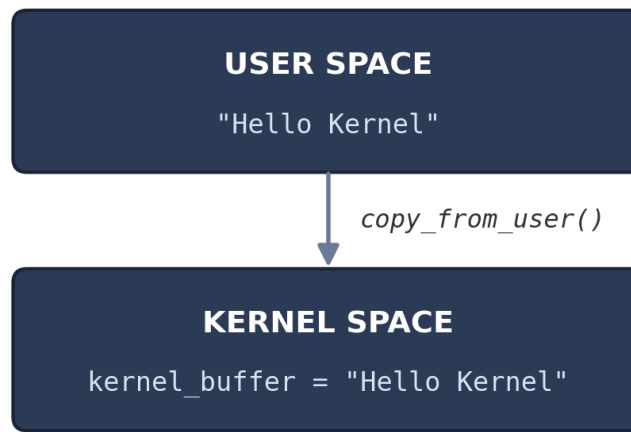


Figure 2 — `copy_from_user()` moves data from user space into the kernel buffer.

Reading: kernel space → user space

When `sudo cat /dev/mychardev` ran, the driver's `my_read()` was called. `copy_to_user()` moved the data back the other way:

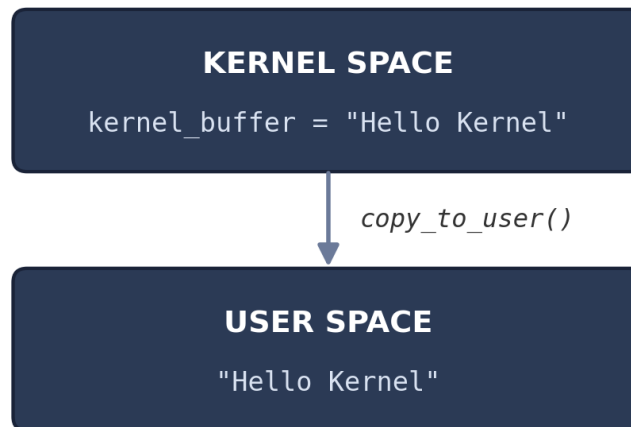


Figure 3 — `copy_to_user()` moves data from the kernel buffer back to user space.

12. The Four Important Functions

These four callbacks are the heart of any character driver:

System call	Driver callback	Called when
<code>open()</code>	<code>my_open()</code>	Called when <code>open("/dev/mychardev", ...)</code> occurs.
<code>write()</code>	<code>my_write()</code>	Called when user space sends data (User → Driver).
<code>read()</code>	<code>my_read()</code>	Called when user space requests data (Driver → User).
<code>release()</code>	<code>my_release()</code>	Called when the device is closed, e.g. via <code>close()</code> .

13. Simple Diagram for the Class

A single picture ties the whole exercise together — from the shell, through the device node, into the driver's callbacks, and down to the buffer that actually holds the data:

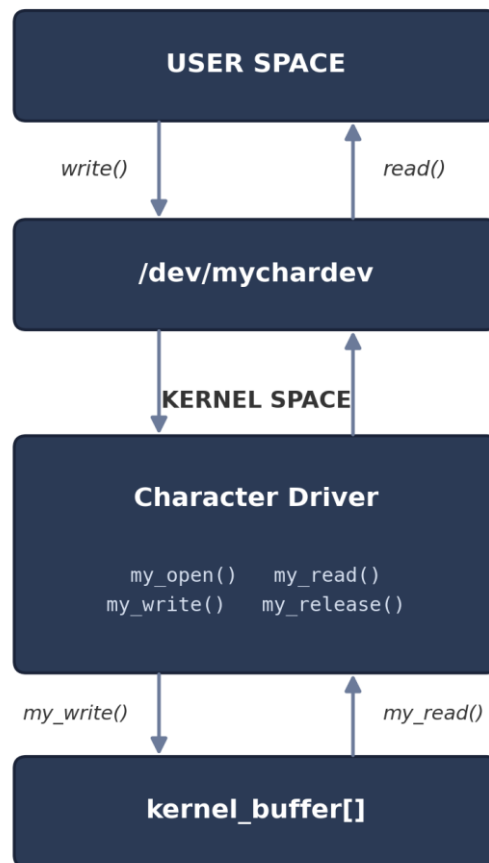


Figure 4 — Full path from a user-space read/write call to the kernel buffer.

In short:

- `write()` → User → Kernel
- `read()` → Kernel → User

14. What Students Learn From This Experiment

After completing this lab, students should understand:

Concept	What it means
Character device	A device that provides a stream-oriented interface for data.
/dev	Linux exposes the device to applications through <code>/dev/mychardev</code> .
Driver callbacks	Linux calls <code>open()</code> , <code>read()</code> , <code>write()</code> , and <code>release()</code> at the appropriate time.
User/kernel boundary	Why <code>copy_from_user()</code> and <code>copy_to_user()</code> are needed.
Device registration	The driver registers itself with the kernel.
Major/minor numbers	Linux identifies the character device using a major number and a minor number.

15. Student Exercises

After the demonstration above, try the following:

Task 1 — Shrink the buffer

Change:

```
#define BUFFER_SIZE 1024
```

to:

```
#define BUFFER_SIZE 64
```

Rebuild and reload the module, then test what happens when you write a message longer than 64 bytes.

Task 2 — Change the boot message

Change:

```
pr_info("mychardev: driver loaded successfully\n");
```

to:

```
pr_info("MY FIRST CHARACTER DRIVER LOADED\n");
```

Reload the module and confirm the new message with:

```
dmesg | tail
```

Task 3 — Send different messages

```
echo "Linux Kernel" | sudo tee /dev/mychardev
sudo cat /dev/mychardev
```

```
echo "Character Driver" | sudo tee /dev/mychardev
sudo cat /dev/mychardev
```

Conclusion

This exercise walked through the full life cycle of a minimal Linux character device driver: registering a major/minor number, wiring up open/read/write/release callbacks, and exchanging data with user space through `copy_from_user()` and `copy_to_user()`. Testing with plain shell commands (`tee` and `cat`) was enough to exercise every callback, and the kernel log confirmed each step. The three student tasks build on this by varying the buffer size, the boot message, and the data sent, without requiring any change to the driver's overall structure.